

FinManager — Opus 4.8 Autonomous Build Brief

You are the lead engineering agent building **FinManager**: a self-hosted, privacy-first, automation-first household finance platform for a Portuguese household (pt-PT, EUR, **Europe/Lisbon**). You lead a **team of sub-agents** and are expected to deliver the **entire application, end to end, without stopping to ask for permission** between steps. Read the references, exercise expert judgment, verify against the rubrics, and keep going until every module's Definition of Done is green.

This brief is self-contained. There is no external "archived master spec" or SRS file in this repository — everything needed to start building, including the shared core domain (§1a below), lives here or in the reference tree below. **Load files on demand, never assume their contents:**

- **modules/** — one enriched, authoritative spec per module (the **primary** per-module source of truth). Read **modules/NN-*.md** in full before implementing that module.
- **seed/** — real fixtures and taxonomy seeds, organised per module (e.g. **seed/supermarket/** holds the pt-PT category taxonomy, eleven real *talões* and the 2025 spreadsheet). Read the module spec for what each file proves. **Precedence:** this brief (including **§1a Shared Core Domain**) wins on process, autonomy, and cross-module conventions → a **modules/NN-*.md** file wins for its own module's detail. If something is unspecified, pick the sensible default, record it in that module's *Open Questions/Decisions* section **and** as a new file under **docs/decisions/** (see §7), and continue — never invent silently, never stall waiting for the user.

0. Prime directive — build continuously, autonomously, to completion

- **Do not stop between phases or modules to ask "should I continue?"** Deliver a tested, demo-able increment each phase and immediately proceed to the next until all nine modules ship and every rubric in §5 is green.
- **Only pause for a genuine blocker:** a destructive/irreversible action, a missing secret you cannot synthesize, a locked-decision conflict, or an ambiguity that materially changes the data model with no defensible default. Otherwise decide, document the decision, and move on.
- **Self-verify before advancing.** A phase is done only when its rubric is green **and** **make check** (lint + types + tests, both apps) passes. If verification fails, diagnose and fix — do not paper over it or disable checks.
- **Leave the tree buildable at every checkpoint.** **docker compose up** must yield a working stack from clean at the end of every phase.

1. What you are building

A single system of record covering supermarket receipts, bank transactions, health reimbursements, utilities, vehicles, net-worth/assets, household multi-user scope, dashboards, and a LEGO collection catalog. It runs entirely on a home Synology DS920+ NAS via **docker compose up**, reachable over LAN/VPN.

The product exists to automate. Data arrives from messy external sources (receipt photos, bank exports, utility PDFs). The system parses, categorizes, reconciles, and enriches it, **auto-accepting high-confidence**

results and routing only the uncertain remainder to a human Review Queue. Users approve and correct; they do not type. **Confidence is visible on every record and the backlog is always in view; the auto-accept rate is reported, never targeted.** Every automated decision is explainable and reversible.

The nine modules (all in scope)

#	Module	Authoritative spec
1	Supermarket & Receipt Processing	modules/01-supermarket.md
2	Bank Statements & Transaction Ledger	modules/02-banking.md
3	Health Expenses & Insurance Claims	modules/03-health.md
4	Household Utilities	modules/04-utilities.md
5	Vehicle & Transportation Management	modules/05-vehicles.md
6	Asset Wealth & Net-Worth Overview	modules/06-assets.md
7	Multi-User & Household Scope	modules/07-household.md
8	Dashboards & Insights	modules/08-dashboards.md
9	LEGO Collection Catalog	modules/09-lego-collection-catalog.md
9.1	LEGO Collection Catalog — usability refinements (no schema change)	modules/09.1-lego-collection-refinements.md
9.2	LEGO Collection Catalog — real inventory, full editing, retirement dates	modules/09.2-lego-real-inventory-and-editing.md
10	First-Run Setup — a clean install configures itself (no schema change)	modules/10-first-run-setup.md

Shared entities used by every module (**User**, **Entity**, **Merchant**, **Category**, **Tag**, **Transaction**, **Document**, **Link**, **ReviewTask**, **AuditLog**, **Setting**, **ImportBatch**, **ProcessingJob**, ...) are canonical and defined **once, below, in §1a** — do not redefine per module. Each module file's **Integration Contract** states exactly what it exposes to and consumes from the others; keep both sides in sync whenever you touch a shared entity.

1a. Shared Core Domain (build in Phase 0 — every other module depends on this)

Identity, household, and RBAC entities (**User**, **Household**, **HouseholdMember**, **Entity**, **Session**) are **fully specified in [modules/07-household.md](#) (M7)** — deliberately the **simplest and first module built**: login, entity/RBAC data model, and the entity selector, in full, in Phase 0. It is intentionally minimal (3 roles, **no per-entity read isolation** — everyone in the household reads everything; **Entity** has no **type** enum, just **name** + **member_ids**; no per-module permission overrides, no invite-token flow, no formal GDPR tooling) — do not re-add that complexity. The remaining cross-module entities have no other home; this is their canonical definition:

- **Merchant** — **id**, **name**, **nif?** (Portuguese tax ID, validated), **kind**(RETAIL|BANK|INSURER|UTILITY_PROVIDER|SERVICE_PROVIDER|OTHER), **default_category_l1_id?**, **default_category_l2_id?**, **aliases** (JSON array),

`website?`, `is_deleted`, `created_at`, `updated_at`. Global reference data (not entity-scoped); every module's `*_merchant_id` FK points here.

- **Category** — `id`, `code_en`, `display_name_pt`, `domain`(GROCERY|BANKING|HEALTH|UTILITY|VEHICLE|OTHER), `level`(1|2|3), `parent_id?`, `brand_axis` (bool, default false), `is_deleted`. Invariants: `parent.domain == child.domain`; `level==1 ⇔ parent_id IS NULL`. Grocery is 3-tier (seeded from [seed/supermarket/categories/supermarket-categories.pt-PT.json](#)); banking is 2-tier; other domains as needed.
- **Tag** — `id`, `household_id`, `name`, `color?`, `is_deleted`. Every module attaches a `tags[]` array of `Tag.id` to its records for cross-cutting labels (`#vacation`, `#daughter`).
- **Transaction** (canonical ledger) — fully specified in [modules/02-banking.md](#); every reconciling module (M1, M3, M4, M5, M9) references `Transaction.id` via its own optional `*_transaction_id` FK — never redefine the ledger row.
- **Document** — `id`, `sha256_hash`, `mime_type`, `byte_size`, `storage_path` (outside web root), `source`(URL|UPLOAD), `url?` (provenance of a copied-from-web file), `signed_url_expires_minutes` (default 15), `created_at`. Owing modules point to it via their own direct FK (`Receipt.document_id`, `LegoSetModel.image_document_id`, ...). Uploads are magic-byte validated before persistence and served only via a signed, time-limited URL — never a static path. Web images are **downloaded once and stored locally**; `url` is kept as provenance and is never re-fetched at render time.
- **Link** — polymorphic reconciliation edge: `id`, `from_type`, `from_id`, `to_type`, `to_id`, `link_type`(RECEIPT_TRANSACTION|CLAIM_TRANSACTION|BILL_TRANSACTION|VEHICLE_EXPENSE_TRANSACTION|OTHER), `confidence`, `decision_reasons` (JSONB), `status`(SUGGESTED|CONFIRMED|DISMISSED), `created_by`, `created_at`. No real FK constraints (polymorphic); compensate with `(from_type, from_id)/(to_type, to_id)` indexes, an app-level allow-list of valid type pairs, and a periodic integrity audit. Supports 1:N and N:1.
- **ReviewTask** (backs the Review Queue) — `id`, `entity_id`, `module`, `subject_type`, `subject_id`, `confidence`, `suggested_payload` (JSONB), `decision_reasons` (JSONB), `status`(PENDING|CONFIRMED|FIXED|DISMISSED), `created_at`, `resolved_at`, `resolved_by`.
- **AuditLog** — `id`, `entity_id`, `actor_user_id`, `action`(CREATE|UPDATE|DELETE|STATUS_CHANGE), `table_name`, `record_id`, `before` (JSONB), `after` (JSONB), `reason?`, `created_at`. Every financial mutation and every lifecycle-status change writes one row here — it is the sole source of truth for reconstructing historical state trends; do not build parallel status-history tables.
- **Setting** — `id`, `scope`(GLOBAL|HOUSEHOLD|ENTITY|MODULE), `scope_id?`, `key`, `value` (JSONB), `updated_by`, `updated_at`. Holds confidence thresholds (default 0.90 auto-accept / 0.60 review, per-source configurable) and every external-provider opt-in toggle + refresh cadence (off by default).
- **ImportBatch** — `id`, `entity_id`, `module`, `source_type`(CSV|OFX|PDF|MANUAL), `file_document_id?`, `status`(PENDING|PROCESSING|COMPLETED|FAILED), `row_count`, `success_count`, `error_count`, `created_at`, `completed_at`.
- **ProcessingJob** — `id`, `idempotency_key` (unique), `job_type`, `entity_id?`, `status`(QUEUED|RUNNING|SUCCEEDED|FAILED|RETRYING), `attempts`, `max_attempts`, `last_error?`, `payload` (JSONB), `created_at`, `started_at`, `completed_at`. Every Celery task is keyed by this row; failures create a row here — never a silent loss.

- **CsvMapping** — bank-import-specific; fully specified in [modules/02-banking.md](#).

All of the above use UUID v7 primary keys, **is_deleted** soft-delete (never hard-delete financial history), and **created_at/updated_at** timestamps unless stated otherwise.

2. Principles to exercise judgment against

- **Automation-first (80/20)**. If a workflow can be automated, automate it and score its confidence. Surface uncertainty; never silently drop or silently guess.
- **Explainable & auditable**. Every automated decision carries machine-readable reasons; every financial mutation is auditable and reversible. If you can't explain a decision in the UI, redesign it.
- **Correctness over cleverness**. Money is a decimal **NUMERIC (10,2)** EUR amount (**_eur** suffix fields), **never a float and never integer minor units/cents**. Ledger writes are ACID and transactional. Migrations are additive and reversible. Historical financial records are **voided, never hard-deleted**.
- **Privacy by design**. Bank Statements and Health (module 2 and 3) have critical information. Any external intelligence provider (LLM/OCR/market lookup) is **opt-in, off by default, clearly surfaced**. Default extraction is 100% local. All other modules can use external intelligence and information providers.
- **Single source of truth**. One normalized relational model. Don't store what you can compute deterministically (except immutable historical snapshots).
- **Portuguese-first, English code**. All code, schema columns, and enum/identifier names are **English**. Domain vocabulary — especially the category taxonomy — carries **pt-PT display names** exactly as the household presented them. UI ships **pt-PT as default locale** and is i18n-ready (no hard-coded strings). Store categories as (**code_en, display_name_pt**).
- **Container-first, always**. The entire stack — backend, frontend, database, Redis, Celery workers, migrations, linters, tests, package managers — runs exclusively inside Docker containers defined in **docker-compose.yml**. **Never install a runtime, package, or tool directly on the host** (no host **pip install/npm install/python/psql/alembic** run outside a container). Every command in this brief, every **Makefile** target, and every CI step is expressed as **docker compose exec/run ...** or a container-wrapped **make** target. The only things allowed on the host are Docker/Docker Compose and the repository source tree.
- **podman**. **podman** and **podman - compose** is your to go container runtime unless higher requirements are needed.

Proposed stack: React 18 + TypeScript + Vite + TailwindCSS + shadcn/ui + TanStack Query/Table + react-hook-form + zod + Recharts + React Router on the front; Python 3.12+ + FastAPI + SQLAlchemy 2.0 + Alembic + Pydantic v2 + Celery + Redis + PostgreSQL 16 behind Caddy, all via Docker Compose. Everything else — file layout, helper design, internal APIs — is yours to design well.

3. Gotchas — spend your attention here

The obvious CRUD is inferable from the module specs. These are the non-obvious traps.

Domain (Portugal-specific)

- **Fs flag**. Fs-flagged receipt items are **excluded from the invoice total** but summed separately into **fs_total_eur / fs_item_count**. Every arithmetic reconciliation must respect this split. Prove it

with unit tests.

- **PVP vs paid.** Keep list price (`_pvp_`) separate from paid price. €/kg has **three** distinct variants the household uses: `pvp` (Price/weight), `promo` ((Price–PromoInd)/weight, individual promo only), and `final` (Price_Final/weight, after the prorated invoice discount too). Don't collapse them.
- **Invoice-level (cartão/global) discount is prorated** across items (`invoice_allocated_discount_eur`), not a per-item promo (`promo_discount_eur`). A common bug is summing per-item discounts and missing the invoice-level credit.
- **Category is intrinsic to a product.** Two items with different categories are, by rule, **different master products**. Enforce `parent.domain == child.domain` and `level==1 ⇔ parent_id IS NULL`. Grocery is 3-tier; banking is 2-tier. Grocery L3 is a normalized product-genus in pt-PT (brand/size-agnostic); brand/variant lives on **MasterProduct**; brand-as-L2 is an accepted exception flagged `brand_axis=true`. Seed: [seed/supermarket/categories/supermarket-categories.pt-PT.json](#).
- **EUR formatting** is comma-decimal (€1.234,56) via a single money util. Money is stored as decimal `NUMERIC(10,2)` EUR (never minor-unit integers) and never reaches the UI as anything but pt-PT-formatted EUR.
- **Dates:** store UTC, but invoice/business dates keep their local calendar date (**Europe/Lisbon**). Don't UTC-shift a receipt's purchase date.
- **Health reimbursements** are multi-source (insurers + mutual funds / ADSE-style): several claims per expense, partial reimbursements accumulate, over-reimbursement ($\Sigma \text{reimbursed} > \text{gross}$) must be flagged, overdue claims (no movement in N days) are "leakage" to alert on. The claim state machine rejects illegal transitions.
- **Utilities:** billing periods overlap or gap — flag them; distinguish estimated vs actual and don't forecast on estimates; gas may be kWh or m³ by region; time-of-use (vazio/cheias/ponta) and contracted power matter.
- **Vehicles:** odometer strictly monotonic per vehicle (flag reversals); `payer_split` sums to exactly 100; L/100km computed **between** consecutive full fills only.
- **Assets:** snapshots immutable (corrections = new snapshot); liabilities reduce net worth; multi-currency valued at the FX rate stored **at snapshot time**, never revalued retroactively. True savings rate = $(\text{income} - \text{expense}) / \text{income}$.
- **LEGO (M9):** deliberately the leanest module — **three tables only** (`LegoSetModel`, `LegoSetInstance`, `StorageLocation`). Catalog identity is split from each owned physical copy so "how many copies do I own" and per-copy tracking coexist. `current_value_eur` is a **single manually maintained field on the model** with a `value_updated_at` stamp — there is no valuation history table, no sealed/used/parts-out bands, no valuation engine, and no automatic price refresh; staleness is surfaced in the UI, not hidden. ROI/appreciation are always **unrealized** (current value vs. cost), `NULL` for gifts (cost 0) and for sets with no value set, and never derived from `sale_price_eur`. `ownership_status` is exactly three states (`IN_COLLECTION|SOLD|GIFTED`); hard delete is offered alongside soft delete. Missing parts are **free text**, not a table, and never adjust value. Storage is a **flat area + container** list (no tree); `capacity_pct` is a hand-annotated 0–100 fullness estimate, never computed from item count. Images: **one per model, one per copy**, always **copied to local storage** (never hotlinked), via the shared **Document**. External marketplace links are **built from a set_number URL template**, never stored. Brickset is the **single** metadata provider (free API key), called only on explicit user action, with a first-class manual-entry fallback. The collection **does not** roll into M6 net worth — by explicit decision.
- **Entity attribution cascades everywhere.** Every record belongs to exactly one **Entity** — a named owner with one or more members (three individuals plus a joint entity for the couple is the expected

shape; there is no **type** enum). Entity is an attribution/filter dimension, **not** a permission boundary: every household member reads everything. RBAC (**OWNER/MEMBER/VIEWER**) governs write power only. Dependents (children) have no login.

Backend / data

- The generic **Link** table is polymorphic (**from_type/from_id, to_type/to_id**) — it **cannot use real FK constraints**. Compensate with (**from_type, from_id**) and (**to_type, to_id**) indexes, an app-level allow-list of valid type pairs, and a periodic integrity audit. Must support 1:N and N:1.
- **UUID v7** for time-sortable PKs (generate in Python before insert).
- **Enums**: prefer lookup tables for externally-visible, evolving enums (**processing_status, link_type**); reserve Postgres **ENUM** for internal, rarely-changing codes.
- **Confidence engine = pure, deterministic functions**. No timestamps/random seeds inside. Returns (**status, confidence, decision_reasons[]**) where each reason is {**rule, detail, score**}. Thresholds (default 0.90 auto-accept / 0.60 review) come from **Setting**, per-source configurable. Learned corrections feed a mappings table **outside** the pure engine.
- **Celery tasks are idempotent** (keyed by **Idempotency-Key** stored on **ProcessingJob**), transactional (no partial ledger states), retry with capped exponential backoff. Failures create a **ProcessingJob** row — never a silent loss.
- **Performance (NFR: <800 ms p95 over 10 years)**: index **receipts(entity_id, purchased_at DESC, id)** and category columns; back dashboards with materialized monthly-aggregate views + Redis cached aggregates invalidated on confirm. Validate with **EXPLAIN (ANALYZE, BUFFERS)** on production-scale seed.

Ingestion / intelligence

- Model providers as protocols — **OcrProvider, ExtractionProvider, SuggestionProvider** — each module registering parsers with a dispatcher that detects document type and routes. Defaults: **pdfplumber** (digital PDF), **pytesseract** (scans), **rapidfuzz** token-set + a learned alias table (suggestions). External LLM/OCR is a swappable, off-by-default provider.
- **Arithmetic reconciliation is a confidence signal**: $\Sigma(\text{non-Fs items}) \approx \text{total_eur}$ and $\Sigma(\text{Fs items}) \approx \text{fs_total_eur}$ within $\sim \pm \text{€}0.02$; mismatch halves confidence with reason **arithmetic_mismatch**.
- **Combine signals** (OCR text, merchant match, product match, arithmetic) into one **[0,1]** score via configurable weights; emit each as a **decision_reason**. Suggested starting weights: product 0.30, OCR 0.25, arithmetic 0.25, merchant 0.20.
- **Normalize Portuguese product text** (strip accents/units/affixes) before **rapidfuzz** token-set ratio; match threshold ~ 0.78 , review band $\sim 0.70\text{--}0.78$. Confirmed corrections append to an alias table.
- **Bank import**: CSV via a reusable column-mapping wizard (saved **CsvMapping**), OFX via **ofxparse**, PDF best-effort. Dedupe on **hash(account, booked_date, amount_eur, description_norm)**; re-import must not duplicate. Detect recurring charges (merchant+amount $\geq 3\times$ in 60d) for change alerts.
- **Reconciliation**: candidate window ± 3 days (configurable) + amount tolerance + merchant fuzzy similarity $\rightarrow$ **Link** suggestions above a confidence cutoff; the rest go to review.
- **Utility anomalies**: seasonal baseline (rolling median per month) + z-score ($> 2\sigma$) or percentage threshold.

Frontend / UX

- **Two architectural pillars:** the global **Entity selector** and the reusable **Review Queue**; everything hangs off them. A third shared component appears early: the **transaction picker** (specified in M9 UX-9.7) that proposes bank transactions near a date with a similar amount — reused by M1/M3/M4/M5/M9 for every "link this to my bank statement" flow.
- **Entity + filter state lives in URL params** (bookmarkable), mirrored to a small store, persisted per user, and is a dependency of every TanStack Query key so switching perspective invalidates the right caches. Show a skeleton on switch, not a full reload.
- **Review Queue is one generic component** consuming `{subject_type, subject_id, module, confidence, suggested_payload, decision_reasons}` with Confirm / Fix / Dismiss. "Fix" inline-edits top uncertain fields and recomputes confidence client-side before submit.
- **The receipt review screen is the crown-jewel flow:** split-pane original image/PDF ↔ parsed line items, uncertain fields visibly flagged (not color-only), inline edits recompute running totals live and warn when $\sum \text{items} \neq \text{total}$ (respecting Fs).
- **Optimistic updates only for single-item edits**, never aggregates. Invalidate the item, the review list, and short-TTL aggregates on confirm.
- **Explainability in UI:** a "Why?" affordance on any auto-classified value showing its `decision_reasons`.
- **PWA:** cached app shell + `staleWhileRevalidate` read views; mutations require online (queue with a visible "syncing" badge if offline). Installable with a receipt-photo share target.
- **Types:** generate from OpenAPI (`openapi-typescript`) in CI; never hand-duplicate the contract.
- **Charts (Recharts):** net-worth `LineChart`; allocation `PieChart` donut; price-per-kg multi-line trend; utilities `ComposedChart` with seasonal `ReferenceArea` bands; correlation `ScatterChart`.

4. Verification rubrics (definition of done)

Treat these as the definition of done and prefer expressing them as executable tests. Do not consider a phase complete until its rubric is green and `make check` passes for both apps.

- **Foundation:** `docker compose up` yields a working stack from clean; login + RBAC; household/entity/merchant/category/tag CRUD; empty dashboard + Review Queue shell; audit log records every mutation.
- **Ingestion quality:** every parsed line and every receipt carries a **visible confidence**, and a status screen shows what is waiting for a human. The auto-accept rate is **reported as a trend, never set as a target** — a threshold invites tuning `confidence.auto_accept` downwards to hit it. Arithmetic reconciliation is enforced, and the Fs rules are proven by unit tests.
- **Explainability:** every auto-decision exposes human-readable reasons in the UI and is user-overridable; corrections feed learned mappings.
- **Correctness:** no float money anywhere; every ledger write transactional; re-import and retry idempotent; snapshots immutable.
- **Reconciliation:** receipt↔transaction, claim↔transaction, bill↔transaction links (1:N and N:1) with confidence, auto-created when confident, else reviewed.
- **Performance:** dashboards <800 ms p95 on 10-year-scale seed (measured, not assumed).
- **Health:** claim state machine rejects illegal transitions; multi-insurer out-of-pocket math correct; overdue alerts fire.
- **Vehicles/Utilities/Assets:** monotonic-odometer + 100%-split guards; overlap/gap + estimated-vs-actual handling; immutable snapshots.

- **LEGO (M9):** unrealized ROI math (incl. gift/zero-cost and no-value-set cases), ownership-transition guards, storage capacity math.
- **Security (OWASP):** parameterized queries only; attachments validated by magic bytes, stored outside web root, served via signed time-limited URLs; CSRF on cookie auth; secrets from env; rate-limited auth/upload; encryption at rest for attachments and backups.
- **Seed realism:** deterministic Portuguese seed (Continente, Pingo Doce, Lidl, Piquete da Fruta, pharmacies, Galp/BP, EDP; pt-PT categories; sample receipts/statements/bills with appended Fs articles and a loyalty discount) that powers the four correlation dashboards. For M1 the seed is **real**: eleven fixtures and a 2,429-row spreadsheet live under [seed/supermarket/](#).

5. Delivery order — ship a tested increment each phase, then continue immediately

0. **Foundation & Household (M7)** — shared core domain (§1a), the full (simple) household/entity/RBAC module — login, member/dependent management, entity selector — migrations, deterministic seed, [make check](#), [docker compose up](#), empty dashboard + Review Queue shell. Household is deliberately first: it's the smallest module and every other module depends on its [entity_id](#)/RBAC contract.
1. **LEGO (M9)** — deliberately built **second**: it is the next-smallest, most self-contained module (three tables, no OCR/ingestion pipeline, no reconciliation engine, no background jobs) and is the fastest way to prove the end-to-end pattern (migration → service → router → UI → tests → docs) before tackling the harder ingestion-heavy modules. Ship the catalog, copies, manual valuation, flat storage locations, locally stored images and the Brickset lookup. It is fully self-contained — its only outward dependency is the optional purchase link to a bank [Transaction](#), wired for real once M2 exists (step 2).
2. **Banking ledger (M2)** — statement import (CSV/OFX/PDF), ledger grid, 2-tier categorization, tagging, cash-flow feeds.
3. **Receipts & OCR (M1)** — ingestion engine, parse/OCR, receipt-review screen, master products, merchant CRUD, price history, receipt↔transaction reconciliation.
4. **Health, Utilities, Vehicles (M3–M5)** — specialized tracking + ledger reconciliation.
5. **Assets & net worth (M6)** — snapshot engine, net-worth/allocation/savings analytics.
6. **Dashboards & PWA polish (M8)** — four correlation views, forecasts, anomaly alerts, offline shell, performance tuning to NFR-2.

Advance through all phases in one continuous effort. Do not stop after a phase to seek approval — verify the rubric, keep the contracts in sync, and start the next phase.

5a. The reuse protocol — run this at the start of *every* module, in whatever order it is built

The delivery order above is the recommended sequence, not a guarantee. A module may be built first, last, or out of order, and every [modules/NN-*.md](#) file is written to be **order-independent**: it declares what it *consumes* in its Integration Contract and never assumes what already exists. Four rules make that work.

1. **Discover before you design.** Before writing a line, take the module's Integration Contract, and for each entity it consumes, check whether it already exists in the tree. Read the code, not your memory of it. Anything already present is used as-is; anything missing is built **to the §1a definition**, never as a module-local variant. Two modules defining their own [Merchant](#) is the single most expensive mistake available here.

2. **Extend, never fork.** If a router, service, model or component already covers a shared concern, extend it. A second merchant endpoint, a second review queue, a second money formatter or a second document pipeline is a defect, not progress — regardless of which module arrives first.
3. **Promote shared components on sight.** When a module needs a component that clearly serves others — a picker, a review surface, a reconciliation widget — build it in the shared layer immediately, even if only one module uses it today. The module spec that first describes such a component says so explicitly; honour that, and do not bury it inside the module's own folder.
4. **Record what you shipped.** Every phase appends to **docs/capabilities.md**: one row per shipped capability with *what it is*, *where it lives*, and *which module shipped it*. That file — not any module spec — is the answer to "what already exists?". Keep it current in the same change that ships the feature. It is what lets a later module skip rebuilding, and it stays correct no matter what order the phases ran in.

Corollary: **module specs must not encode build order or file paths.** No "built fourth", no "already seeded in Phase 0", no **app/services/foo.py** references. Those go stale the moment the order changes or the code moves. Dependencies belong in the Integration Contract; locations belong in **docs/capabilities.md**.

6. Documentation & decision records (the project's wiki)

- **Root README.md** must always be current and cover: what FinManager is, prerequisites (Docker + Docker Compose — nothing else), **how to start the stack** (**docker compose up** / relevant **make** targets), how to seed demo data, how to run **make check**, how to tail logs / open a debugger per service, how to reach each service (app URL, API docs, Caddy, etc.), and how to stop/reset the stack.
- **docs/ wiki**, kept current alongside the code (update it in the same change that ships the feature, not as a follow-up):
 - **docs/architecture.md** — service topology, container map, request flow.
 - **docs/capabilities.md** — the running inventory described in §5a: what exists, where it lives, which module shipped it. Append-only in practice; corrected when something moves. Every phase updates it.
 - **docs/decisions/** — one short ADR-style file per non-obvious decision (**NNNN-title.md**: context, decision, consequences), append-only, never rewritten after the fact. This **is** the "record it and continue" mechanism referenced throughout this brief and in each module's *Open Questions/Decisions* section — the module file states *what* was decided, the ADR states *why*.
 - **docs/database.md** — how to open a shell/GUI against the containerized Postgres (e.g. **docker compose exec db psql ...** or a containerized Adminer/pgAdmin service), the schema map, and how migrations are organized.
 - **docs/debugging.md** — how to tail/follow logs per container, attach a debugger, inspect Celery/Redis queues, and read **ProcessingJob/AuditLog** rows for a failed operation.
 - **docs/testing.md** — see §8.
- A phase is not done if **docs/** and the root **README.md** don't reflect what was actually built.

7. Testing discipline — critical paths only, clearly identified

- **Do not chase coverage for its own sake.** Write automated tests only for the few things that are genuinely load-bearing and easy to get subtly wrong: money/decimal arithmetic (never floats), the Fs valuation rules, immutability of snapshots, idempotent re-import/retry, the confidence engine's pure scoring functions, and lifecycle state-machine guards (health claims, LEGO ownership transitions, odometer monotonicity, 100%-split validation). Everything else is validated by manual/exploratory verification against the module's Definition of Done — don't hand-write exhaustive CRUD tests.
 - **Tests must be clearly identified and discoverable**, not scattered ad hoc:
 - Backend: `apps/api/tests/unit/` (pure domain logic, no I/O) and `apps/api/tests/integration/` (DB/Celery-backed, run inside the container against the test database), one file per module (`test_lego_roi.py`, `test_receipt_fs_split.py`, ...).
 - Frontend: co-located `*.test.tsx` next to the component, plus `apps/web/tests/e2e/` for the one or two crown-jewel flows (receipt review, review queue confirm) if e2e tooling is set up.
 - Every test file's header states **which rubric bullet or gotcha it protects against**, so `make check`'s test output doubles as a rubric checklist.
 - `make check` runs the full (small, curated) suite **inside containers only** and is the sole gate; never disable or skip checks to make the tree look green.
-

8. Guardrails

Do not: install or run anything directly on the host machine (backend, frontend, database, Celery, migrations, linters, tests — everything runs via `docker compose exec/run`); use floats for money; hard-delete financial history; swap a locked technology; break an Integration Contract without updating both sides; enable an external provider by default; ship code that fails `make check`; or halt the build to ask for permission on a decision that has a defensible default. Beyond these, prefer sound engineering judgment — read the references, decide, verify against the rubric, record open decisions in the module spec and `docs/decisions/`, and keep building until FinManager is complete.
